

The Simunto logo consists of the word "Simunto" in a white, rounded, sans-serif font, centered within a dark purple rounded rectangle.

Simunto

# DSim

A distributed mobsim ready to use

Christian Rakow, Janek Laudan · 2026 · Simunto GmbH

# Recap

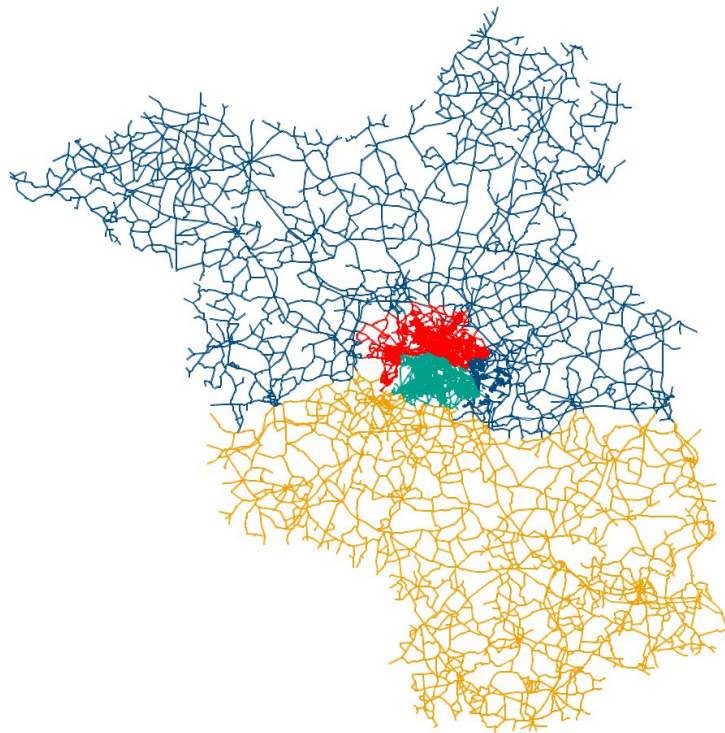
# Recap – Architecture

Distribute Workload geographically

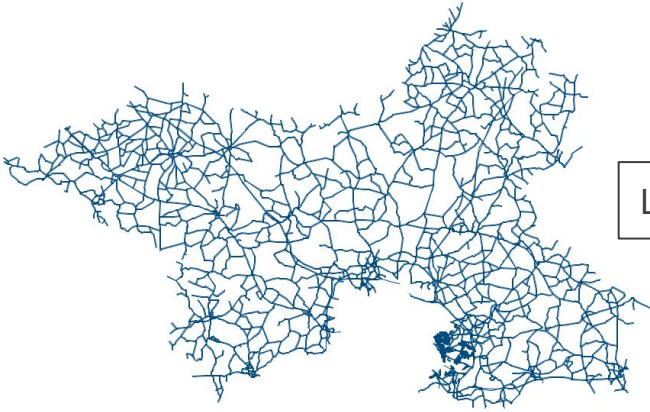
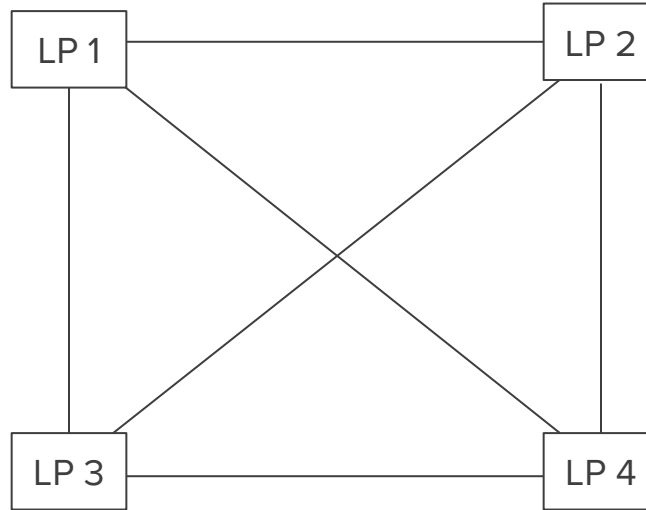
Domain Decomposition is a well-understood problem with plenty of algorithms, e.g., METIS

Estimate computational load for each vertice of the graph by estimating #vehicles crossing the node

METIS balances the computational load across partitions

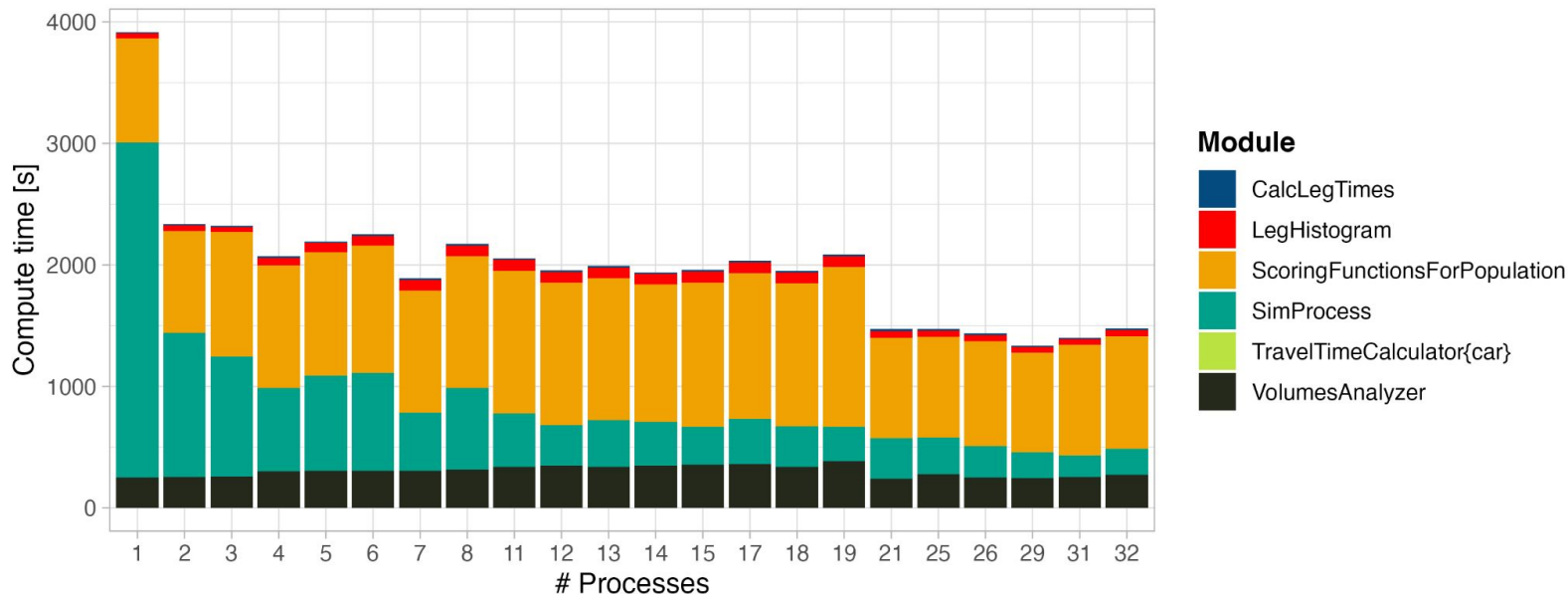


# Recap – Architecture



# Scaling limitations

Global scoring event handler dominated compute time (on partition 0)



# Incompatible Modules

**Modifying global state is not feasible in a distributed simulation**

- Scoring
  - Uses one global event handler which collects almost all events produced by mobsim
- EV Contrib
  - Has a global map of charging state which event handlers freely mutate
  - Charger Reservations are written into a global state as well
  - WithinDayEvEngine accesses global agent state and mutates plans
- SBB Transit QSim Engine
  - Not partition aware

# Contributions

# Backpack Scoring

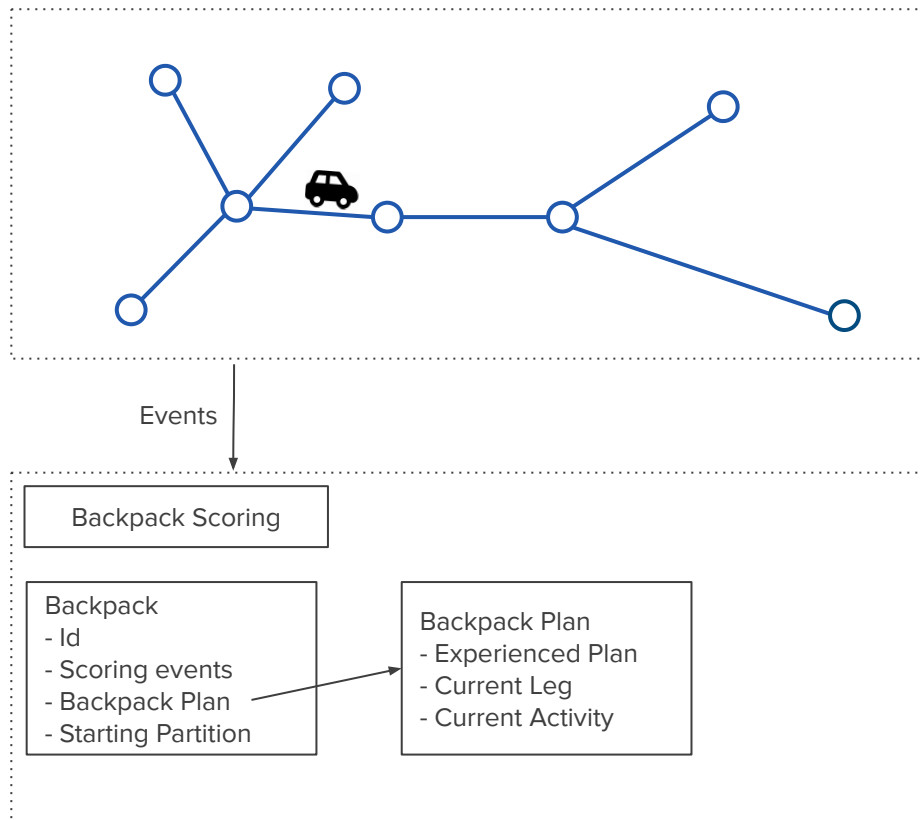
Scoring is based on experienced plans

Agents assemble their experienced plans while moving through the simulation

Backpack Scoring collects events from the simulation and dispatches them to the corresponding backpack

Backpack contains

- Experienced Plan
- Intermediate representation of the current plan element that produces a plan element on finish



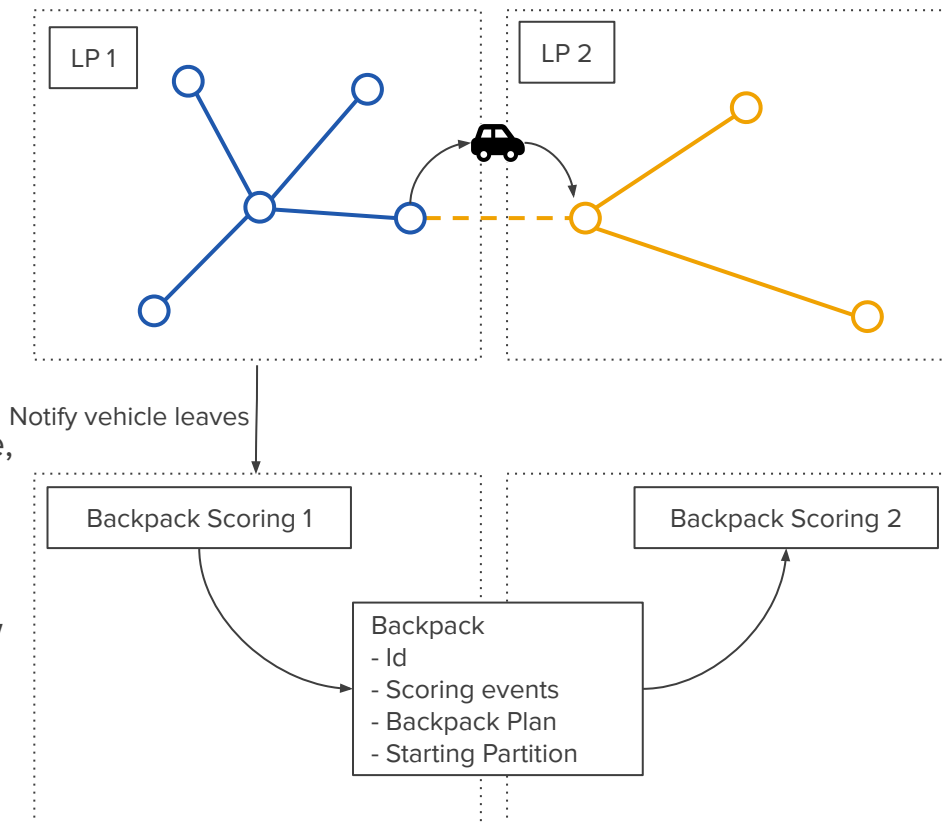
# Backpack Scoring

The backpack is transferred between partitions along with its agent

Events can be collected and processed locally on each partition

Scoring functions are called in the scoring phase, once the entire experienced plan is assembled

- **TripScoring** is encouraged
- **PlanScoring** could be added easily now



# Changes to scoring

**ArbitraryEventScoring** has been deprecated

- Collecting arbitrary events and transferring them is not feasible for DSim
- Antipattern anyway
- Use custom event handler and dispatch **PersonScoreEvent** or **PersonMoneyEvent**, which are handled by Backpack Scoring
- **CharyparNagelLegScoring** uses **TripScoring** now instead of **ArbitraryEventScoring**, for example

# Things we added

New APIs **NotifyAgentPartitionTransfer** and **NotifyVehiclePartitionTransfer**

- DSim notifies implementers when agents or vehicles are transferred
- Implementers react to notification and transfer state as well

New Infrastructure: **PartitionTransfer**

- Accessible via injection
- Engines can use it to transfer their state

# Things we added

For better scalability, slack in event handling is needed:

- Asynchronous execution
- Partitioned Event Handlers
- Direct execution

New annotation

`@DistributedEventHandler`

One event handler per Simulation, DSIm takes care of synchronization, handler is invoked while the simulation progresses as separate task. Suitable for online analysis code.

```
@DistributedEventHandler(blocking = BlockingMode.ASYNC)
public class VolumesAnalyzer ...
```

One event handler per process, DSIm takes care of synchronization, handler is invoked directly from the simulation process, no scheduling overhead, perfectly in sync with the simulation. Only events from one partition are received. Suitable for code that alters simulation state.

```
@DistributedEventHandler(value = DistributedMode.PARTITION,
processing = ProcessingMode.DIRECT)
public class DriveDischargingHandler
```

# Contribs use new APIs too

## EV Contrib

- ~~• Has a global map of charging state which event handlers freely mutate~~
- Uses **NotifyVehiclePartitionTransfer** and **PartitionTransfer** to transfer charging state

```
// Distributed ElectricFleet implements NotifyVehiclePartitionTransfer
@Override
public void onVehicleLeavesPartition(DistributedMobsimVehicle vehicle, int toPartition) {
    var ev = electricVehicles.remove(vehicle.getId());
    var msg = new ElectricVehicleMessage(ev.getId(), ev.getBattery().getCharge())
    partitionTransfer.collect(msg, toPartition);
}
```

# Contribs use new APIs too

## EV Contrib

- ~~Charger Reservations are written into a global state as well~~
- Charger Reservations are managed locally – message based communication in between partitions to coordinate reservations
- ~~WithinDayEvEngine accesses global agent state and mutates plans~~
- WithinDayEvEngine registers as **Activity**- and **DepartureHandler** and modify agent state while they have ownership of an agent

# Manage Agents

## SBB Transit QSim Engine

- ~~Not partition aware~~
- Transfers managed agents in between partitions using **PartitionTransfer**
- Notifies DSim about transfers

```
if (!partitionTransfer.isLocal(e.linkId())) {  
    var veh = (DistributedMobsimVehicle) e.context.driver.getVehicle();  
    var toPartition = partitionTransfer.getPartitionIndex(e.linkId());  
    partitionTransfer.collect(new TransitEventMessage(e, veh), toPartition);  
  
    internalInterface.notifyAgentLeavesPartition(e.context.driver, toPartition);  
    for (var passenger : veh.getPassengers()) {  
        internalInterface.notifyAgentLeavesPartition((DistributedMobsimAgent) passenger, toPartition);  
    }  
}
```

# Benchmarks

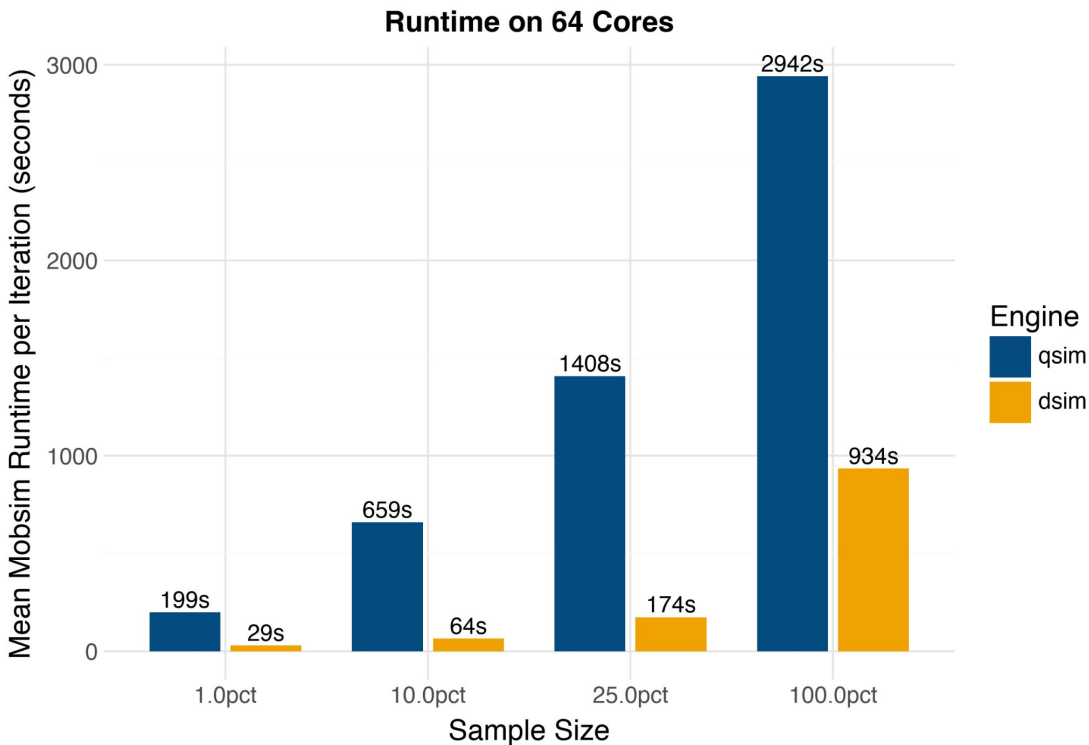
# It is fast!

Benchmark with Simunto's Switzerland scenario

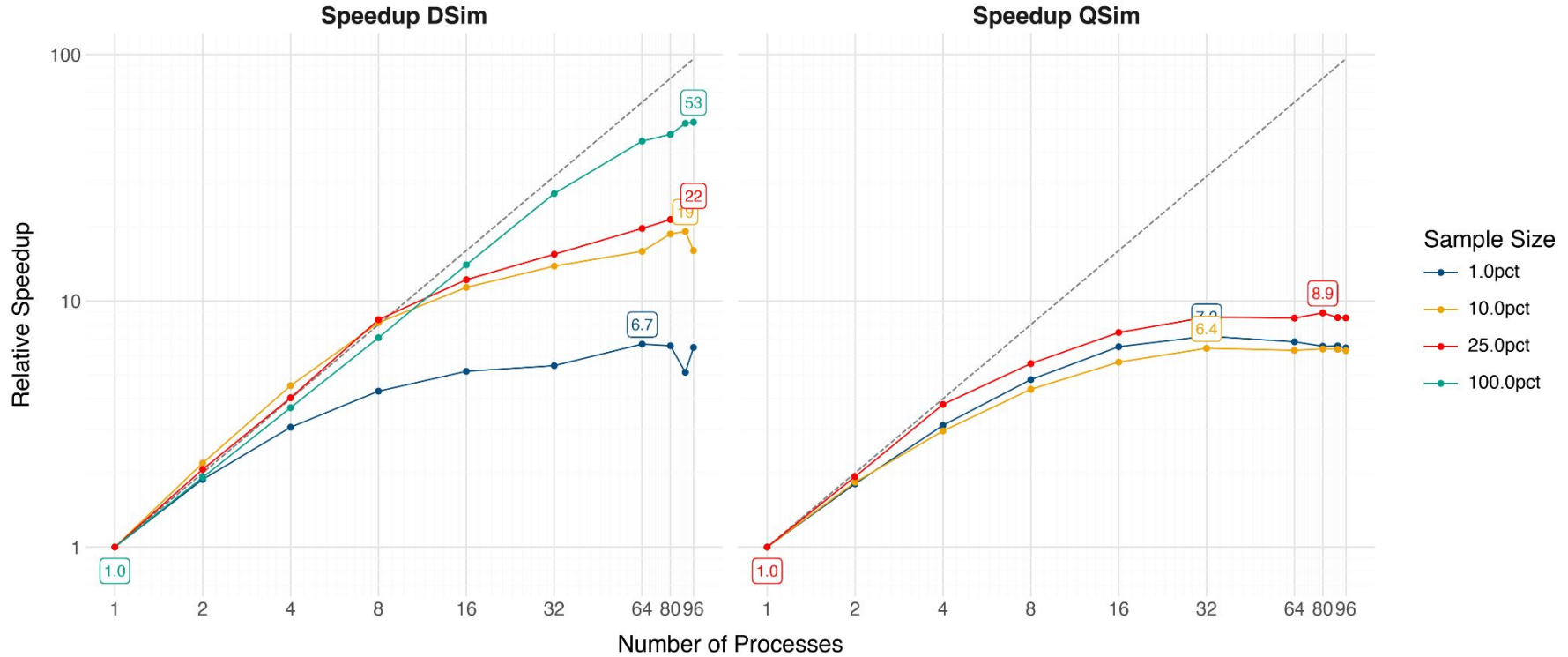
- ~ 10,000,000 agents
- ~ 1,000,000 links in the network

Executed at NHR@ZIB on AMD EPYC 9654 CPU with 96 cores

DSim is 3 - 8 times faster compared to QSim using 64 processes.

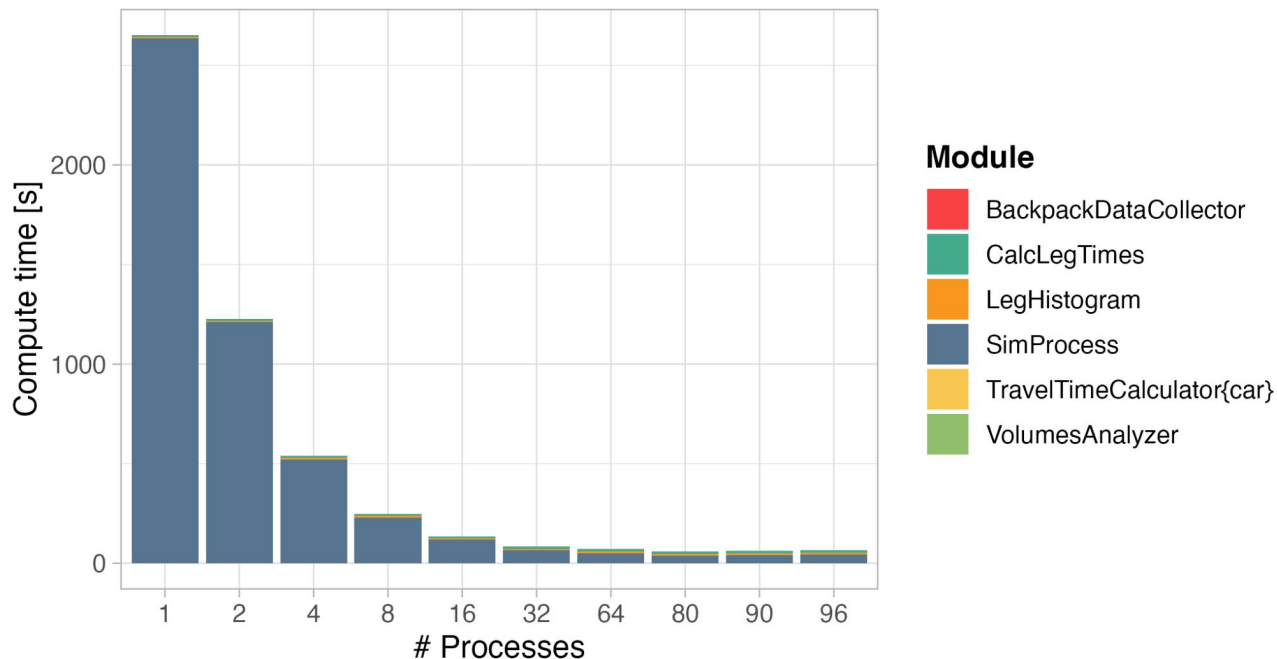


# It scales!



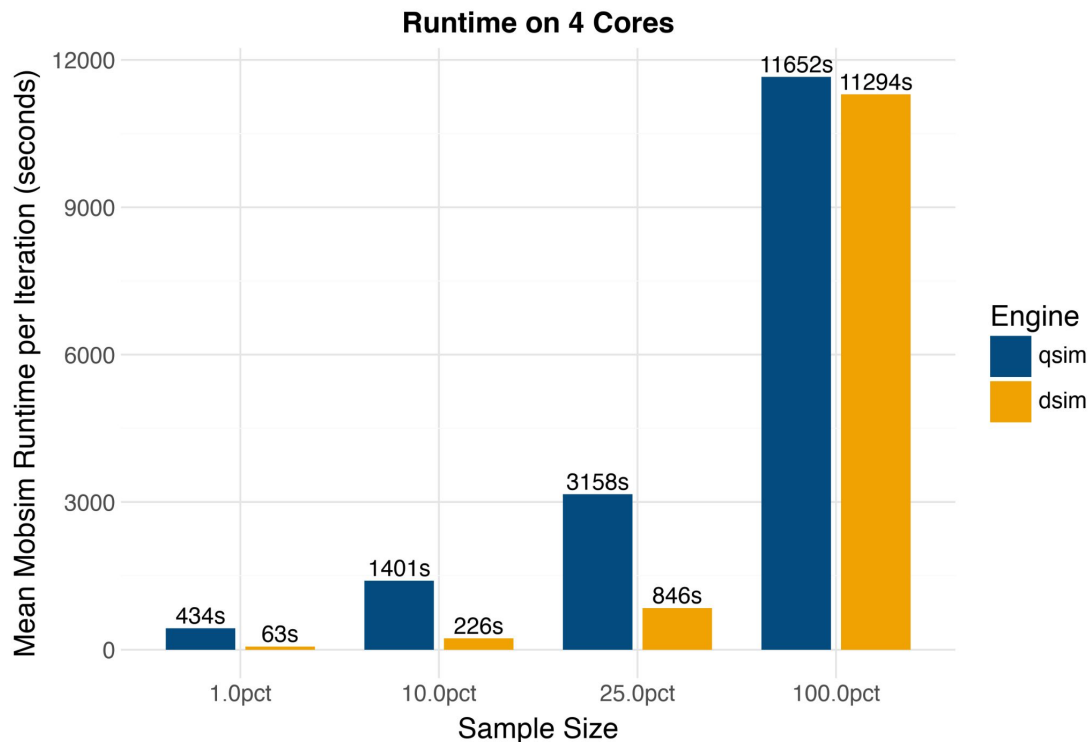
# No more event-handling bottlenecks

Summed durations on Partition 0 (25% sample)



# It is fast (even with few cores)

DSim is also faster when executed on a small machine



# It is ready!

DSim is part of the MATSim core

- Configurable via config.xml
- Add dsim as mobsim to controller module
- Add dsim config module with similar parameters as qsim

No extra dependencies necessary  
(as of PR-5167)

For optimal results METIS must be available on the system. Fallback algorithm bisect works without dependencies

```
<module name="controller">  
  <param name="mobsim" value="dsim" />  
</module>
```

```
<module name="dsim">  
  <param name="networkModes" value="car,bike" />  
  <param name="partitioning" value="metis" />  
  <param name="threads" value="48" />  
  <!-- more parameters available -->  
</module>
```

# Conclusion

1. DSim is ready to use
2. Faster execution times by switching your config
3. APIs for MATSim developers have landed
4. Many contribs are DSim-ready

# Outlook

Help and Support appreciated

- Support for multi node setups can be improved
- Adoption in production model setups
- User-Feedback regarding stability and API design

# Thank you for your Attention

Dr. Christian Rakow  
Software Consultant

<https://www.linkedin.com/in/christian-rakow/>

Dr. Janek Laudan  
Software Consultant

<https://janeklaudan.com>

<https://www.linkedin.com/in/janek-laudan-8a5237228/>

Simunto GmbH  
Riedgrabenweg 49  
8050 Zürich  
Schweiz  
[mail@simunto.com](mailto:mail@simunto.com)

Parts of the presented work were supported by

**VOLKSWAGEN GROUP**

# Make your event handler scale

One event handler per Simulation, DSim takes care of synchronization, handler is invoked while the simulation progresses as separate task. Suitable for online analysis code.

```
@DistributedEventHandler(blocking = BlockingMode.ASYNC)
public class VolumesAnalyzer ...
```

One event handler per process, DSim takes care of synchronization, handler is invoked directly from the simulation process, no scheduling overhead, perfectly in sync with the simulation. Only events from one partition are received. Suitable for code that alters simulation state.

```
@DistributedEventHandler(value = DistributedMode.PARTITION, processing
= ProcessingMode.DIRECT)
public class DriveDischargingHandler
```

One event handler per JVM, invoked directly from the simulation process, no scheduling overhead, handler must take care of synchronization

```
@DistributedEventHandler(value = DistributedMode.NODE_CONCURRENT,
processing = ProcessingMode.DIRECT)
public final class TravelTimeCalculator
```

# Recap – Event Handling

Event handlers are executed in the same thread pool as the simulation

Threadpool executes all tasks (event handlers & simulation processes) for one sim step

For backwards compatibility handlers are called as global instance and events are synced per time step

